The Hong Kong University of Science and Technology
COMP 2611: Computer Organization
Spring 2026

Programming Project: Minesweeper

Due: 14th May, 2026, 2359 HKT

Contents

1. Introduction

Minesweeper is a classic puzzle video game. In the game, mines are randomly placed on an board with no revealed tiles. The player's task is to use mouse clicks to reveal all tiles which do not have a mine by using the clues about the number of neighbouring mines in each tile.

While the game is quite simple to play, coming up with a good strategy boils down to familiarity with the game. Thankfully, you do not need to master the game in this project; we are only going to implement the logic behind a typical minesweeper game.

Most of the rules in our game are similar to typical online versions. While you can play the game with the link to get a feel of how the game operates, we will introduce the rules in detail in the next section for clarity.

This project requires **Java 17** and our modified version of MARS. Links to how to install JDK 17 are included in the appendix.

2. Game Rules

The game is played on a 16 by 16 grid, with 40 placed mines. This mimics a typical "medium" sized minesweeper setup. To simplify your task, we will not ask you to implement support for grids with multiple sizes.

We will be adopting the zero-indexed $(\text{row}, \text{column})$ notation throughout this description. That is, coordinates will be row first, followed by the column. That is:

- The top left corner is $(0, 0)$;
- The tile to the right of the corner is $(0, 1)$; and
- The tile below the corner is $(1, 0)$.

2.1. Initial Game State

The following figure shows the initial state of our minesweeper game.

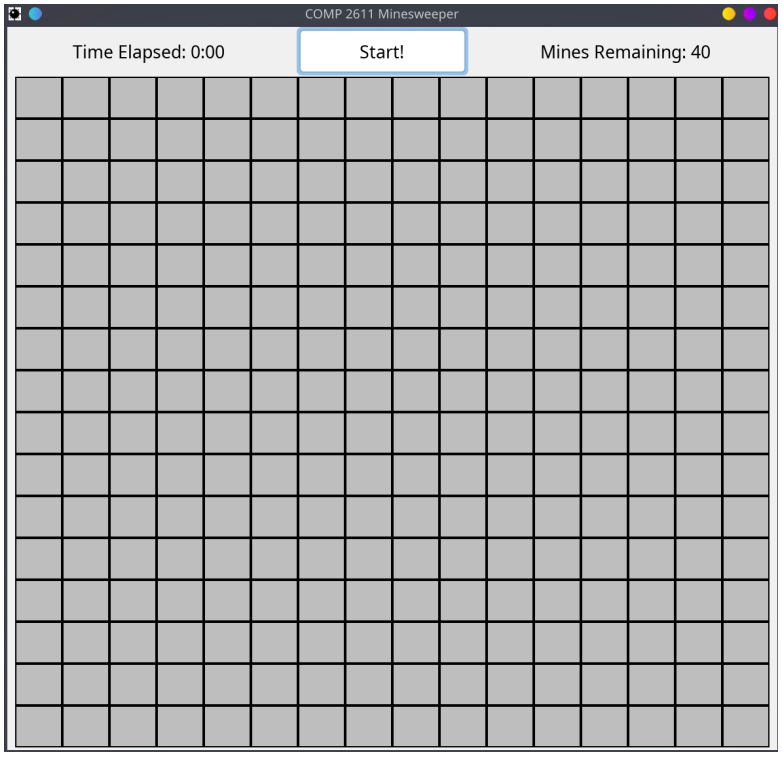

Figure 1: Initial Minesweeper State

The window consists of the timer, the start button, the mine counter, and the game grid. The timer shows the time elapsed for your current game. In general, a lower time elapsed time is better as you can solve the puzzle faster. The mine counter shows the number of mines remaining. You can decrease this count by using the flag feature (see below).

Initially, you should not be able to interact with the game grid as the game has not been started. To start the game, click the "Start!" button. This will start the timer and enable you to start clicking on the tiles in the grid.

After making a guess by left-clicking on a random tile, there are two possible outcomes: you might hit a mine and lose, or you have successfully revealed some number of tiles.

2.2. Revealing a Tile

If you did not hit a mine, your UI might now look like this:

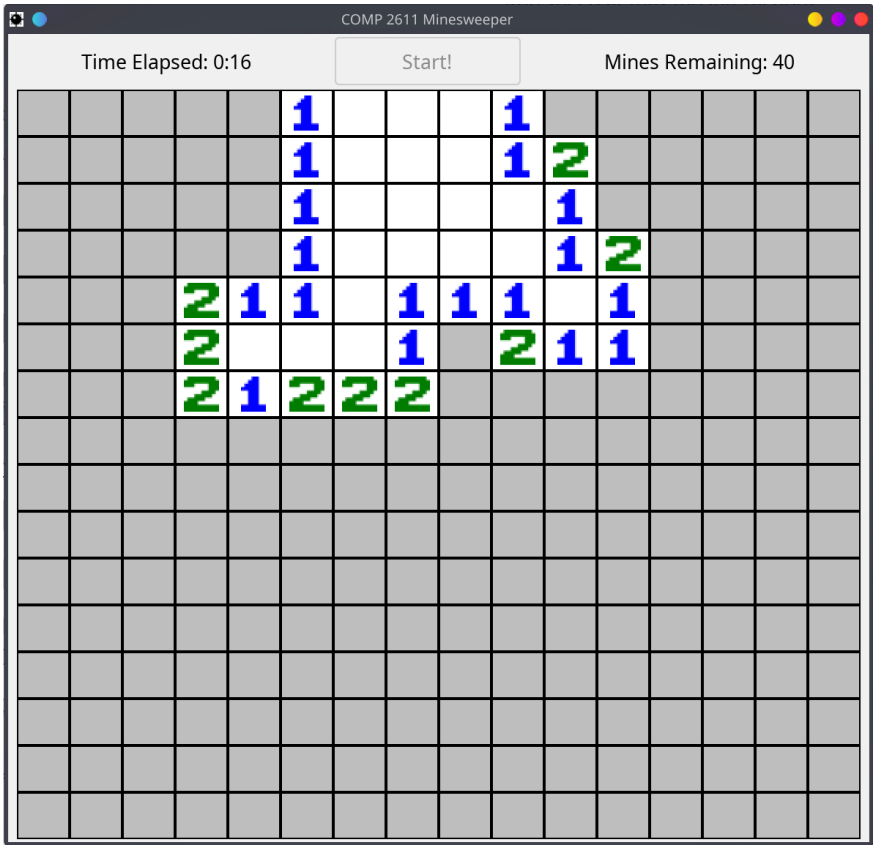

Figure 2: Minesweeper UI After Left Click

White tiles denote revealed tiles, and gray tiles are tiles which have not yet been revealed. Each number on a revealed tile denotes the number of neighbouring mines in the adjacent 8 tiles. For example, seeing a number 2 means that exactly two neighbouring tiles have mines. Revealed tiles with no mines will not have any numbers on them.

If you clicked on a non-mined tile, the tile itself will be revealed. Addition, if the tile has no adjacent mines, all of its adjacent tiles will be revealed. This process is recursive, meaning that it will continue to reveal additional tiles starting from the adjacent tile(s) if the adjacent tile(s) have no neighbouring mines by themselves.

As an example, clicking $(0, 6)$, $(5, 4)$ or $(4, 10)$ on the board above should yield the same results. You can check the next page for a figure with these tiles highlighted in red.

Note that clicking on a tile which is already revealed or has a flag has no effect.

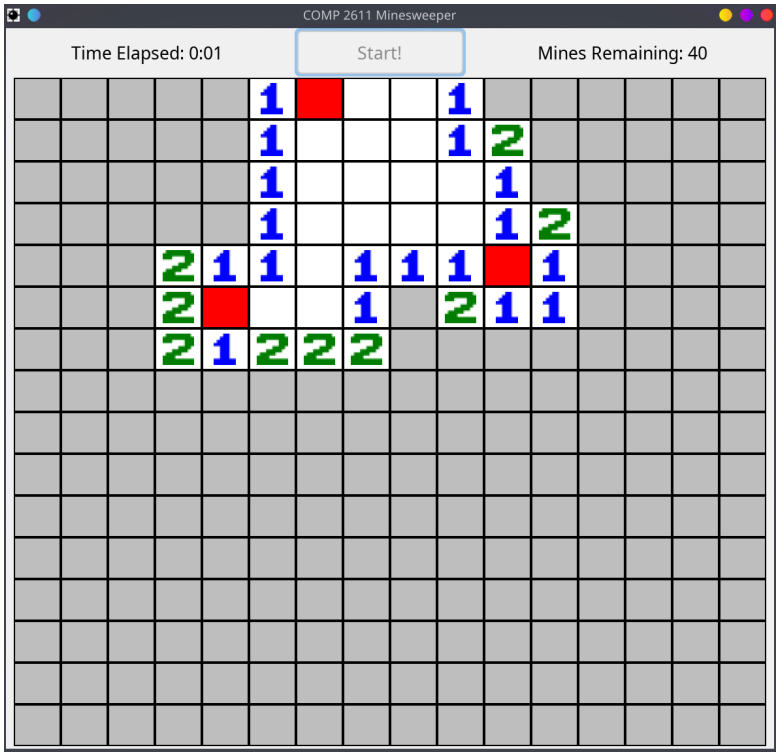

Figure 3: Minesweeper UI After Left Click with Highlighted Tiles

2.3. Hitting a Mine

If you hit a mine, you should see the following UI:

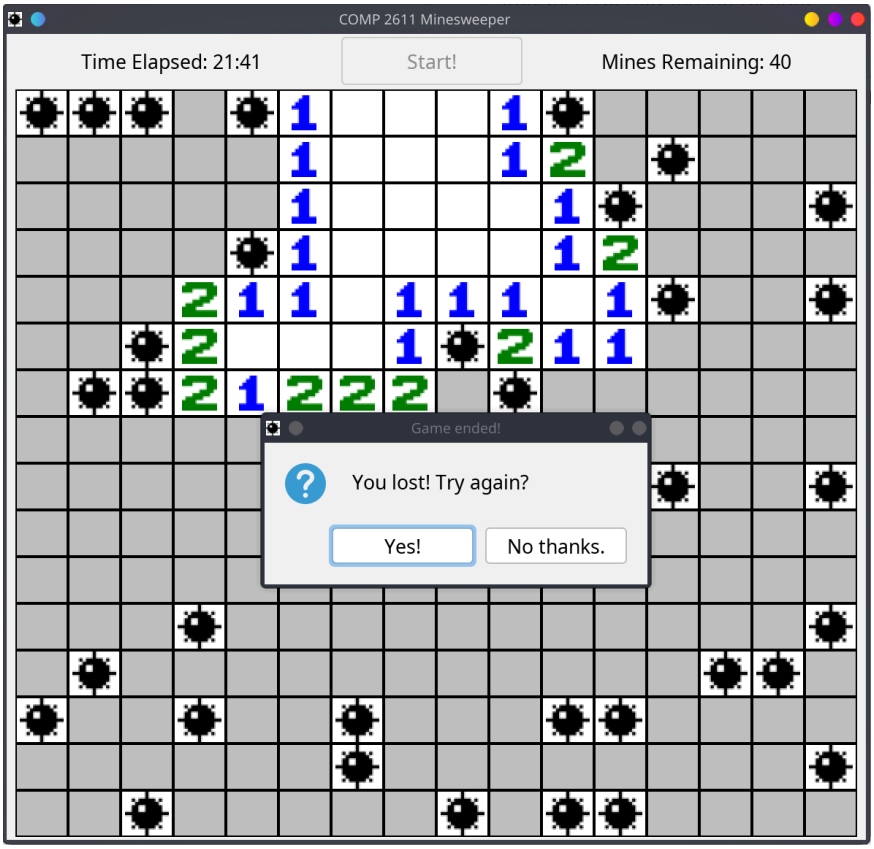

Figure 4: Minesweeper UI After Left Click

At this point, the player has lost as they are not supposed to click on the mines. All mines on the grid are revealed, and a dialog box with the option to retry or quit is displayed. This will also stop the in-game timer.

2.4. Setting Flags

Sometimes, you can infer that there is a mine at a specific tile. To both remind yourself and to avoid accidentally clicking on it, you can add a flag to the given tile by right-clicking on an unrevealed tile. This will decrement the "Mines Remaining" counter to help you keep track of how many mines there are remaining.

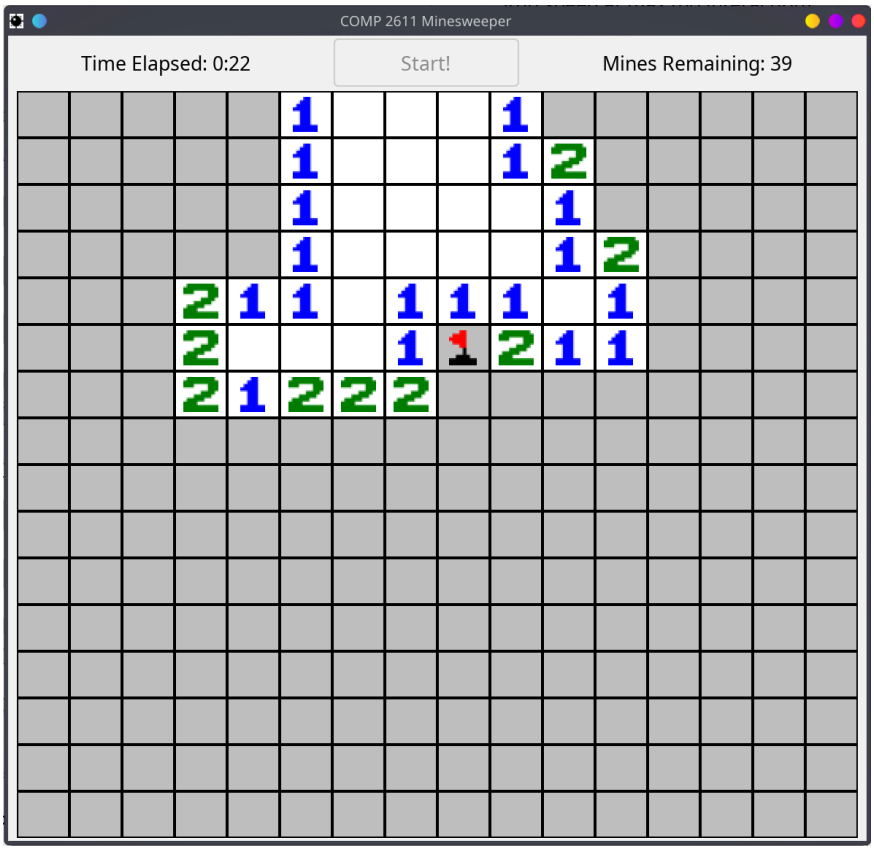

Figure 5: Minesweeper UI With Flag

The image above shows an added flag on $(5, 8)$. Note that after adding a flag, you cannot reveal the tile without first removing the flag.

To remove a flag, you can right-click on the flagged tile again.

2.5. Winning the Game

There are two ways to win this minesweeper game. You can either:

- Reveal all non-mined tiles; or
- Place a flag on exactly where all mines are.

The two images below show different win conditions: the first image shows the "all non-mined tiles revealed" win condition. Although one of the tiles $((11, 3))$ have not been revealed, all non-mine times have been revealed.

The second image is a more extreme case. Although no tiles have been directly revealed, all mines have been marked by a flag. Note that if there are extra flags, or flags in the wrong position, the player cannot win with this win condition.

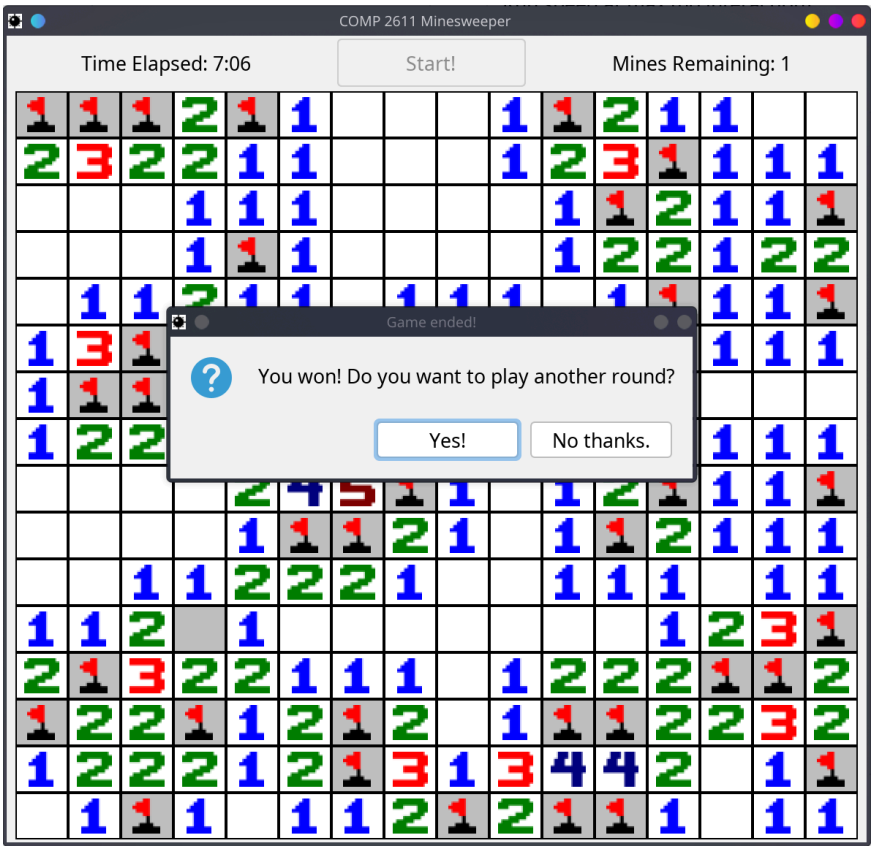

Figure 6: Winning by Revealing all Tiles Without Mines

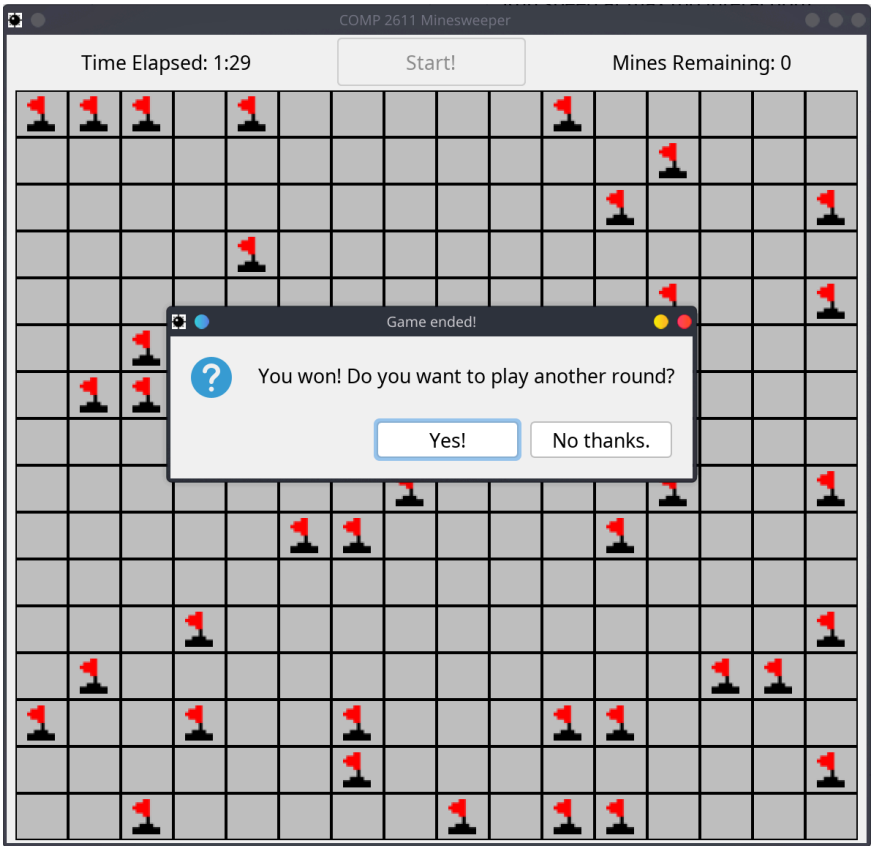

Figure 7: Winning with Flags Only

3. Code Structure

The skeleton code provided consists of two sections: the data segment (for storing the global data members) and the text segment (for our actual program code).

You are encouraged to quickly read through the code to get a good grasp of how it functions, but you do not need to understand every line.

3.1. Data Members (`.data`)

You are provided with the following data members:

Data Member	Data Type
`tiles`	Array of `word` (length 1024)
`action`	`word`
`rowClicked`	`word`
`colClicked`	`word`
`curStatus`	`word`

3.1.1. `tiles`

`tiles` is a 1D array of `word` type. Each tile in the game is described by 4 words of memory space ($4 \cdot 4 = 16$ bytes) in the `tiles` 1D array. Therefore, the length of the array is 16 (rows) $\cdot$ 16 (columns) $\cdot$ 4 (words) $=$ 1024 (words) (or 4096 bytes). Much like Homework 3, Question 2, the tiles are laid out in row-major order. That is, the first 4 words of the array refer to the data of the tile at $(0, 0)$, the next 4 words refer to the data at $(0, 1)$, and the 65th to 68th words refer to $(1, 0)$.

The four words are used as follows:

Word	Internal Name	Usage
0	`row`	The row of the tile.
1	`col`	The column of the tile.
2	`neighbouringMines`	The number of mines in the neighbouring 8 tiles.
3	`bits`	See below.

You might notice that the last word is called `bits`. This is because we store 3 boolean flags in the word to save storage space. Since a word is 32 bits, we will only use the leftmost 3 bits to store the required flags; the other 29 bits are ignored. The last three bits are, in order:

Bit Index (From the Left, 0-indexed)	Internal Name	Usage
29	`isRevealed`	Whether the tiles is revealed.
30	`hasMine`	Whether there is a mine on the tile.
31	`hasFlag`	Whether there is a flag on the tile.

To help visualize this, here are two examples:

`isRevealed`	`hasMine`	`hasFlag`	Bit Values
`True`	`True`	`False`	00000000 00000000 00000000 00000110

isRevealed	hasMine	hasFlag	Bit Values
False	True	False	00000000 00000000 00000000 00000010

3.1.2. action

The action word stores whether the user has inputted a left click or a right click.

0 is used to indicate a left click, and 1 is used to indicate a right click. Middle mouse button inputs are not supported.

3.1.3. rowClicked and colClicked

The rowClicked and colClicked words store the row and column of the tile currently clicked by the user. Only the last click of the user will be stored here.

3.1.4. curStatus

The curStatus word stores the current game state. It is used to indicate whether the game is lost, has won, or is still ongoing.

curStatus	Game State
0	Lost
1	Won
2	Ongoing

Usually, the value of curStatus is updated as the last statement(s) of one of the click handlers.

3.2. Program Code (.text)

The provided skeleton code makes heavy use of different procedures. The implementation of a few procedures are given to you to help simplify your code; other procedures need to be completed by you over the course of the project.

3.3. Helper Procedures

You are provided with the following three helper procedures:

Procedure	Arguments		Return Value ($v0)	Usage
	$a0	$a1		
setSeed	-	-	-	Sets the seed of the random number generator.
getTileAddr	The row of the tile.	The column of the tile.	The computed base address of the tile.	Gets the base address of the tile in the tiles array.
getBit	The value (NOT the address) of the word containing the boolean flags.	The zero-indexed bit counting from the right (if you want the 1 in 00000100, pass 2).	The extracted bit.	Extracts a specific bit from the boolean flags word.

3.4. Code Flow

In the following pseudocode segment, `syscall(n)` denotes a syscall with number `n`.

The main flow is as follows:

Algorithm 1: Minesweeper Logic

```
1:  procedure RunMinesweeper()
2:      ▷ Sets the random seed. Feel free to modify the seed value for testing.
3:      setSeed()
4:      ▷ Initializes the game window.
5:      syscall(200)
6:      ▷ Render board updates.
7:      syscall(201)
8:
9:      while Not Quit Game do
10:         ▷ Task 1
11:         initBoard()
12:         ▷ Task 2
13:         addMines()
14:         ▷ Task 3
15:         updateNeighbourCount()
16:         ▷ Render board updates.
17:         syscall(201)
18:
19:         while Game is Ongoing do
20:             ▷ Wait for input.
21:             syscall(202)
22:             ▷ Get Click Type
23:             syscall(203)
24:             ▷ Get Click Position
25:             syscall(204)
26:
27:             if Left Click then
28:                 ▷ Task 4 & 5
29:                 handleLeftClick()
30:             else
31:                 ▷ Task 6
32:                 handleRightClick()
33:             end
34:             ▷ Render board updates.
35:             syscall(201)
36:         end
37:         ▷ Ask if continue
38:         syscall(205)
39:     end
40: end
```

Note that this is NOT exactly the same as the MIPS logic. However, it provides a reasonable overview of the game logic. Please refer to the main game loop in MIPS for more details.

A detailed list of syscalls provided can be found in appendix B.

4. Programming Tasks

There are six main functions in this project, from which you are required to complete three. They are arranged in rough order of complexity and logic flow of the overall program.

4.1. `initBoard`

In this function, the program (re-)initializes the `tiles` array such that it contains the correct values of for each tile. In particular, `row` and `col` are initialized to the appropriate value, and the remaining words are zero-initialized.

The array is reused across different retries of the game. This function ensures that no data is left over from previous game sessions.

4.2. `addMines`

In this function, 40 mines are added to the game board in a loop. If a mine is placed at a position which already has a mine, the function retries until a mine is successfully placed.

The random position is generated first by row, then by column using syscall 42. Some sample maps have been provided in the appendix for your debugging.

`neighbouringMines` for adjacent tiles are not updated when placing mines; this is the job of the next function. However, the `hasMine` boolean flag has already been set appropriately when this function completes.

4.3. Task 1: `updateNeighbourCount`

After placing the mines in the previous function, you now need to update the number of neighbouring mines per tile to ensure that the game runs correctly.

If a tile already has a mine, you should not update its neighbour count; in general, we consider tiles with mines to have no neighbours.

4.4. Task 2: `handleLeftClick`

In this task, you should handle the behaviour of a left click on a given tile. You may assume that the clicked position is stored in `rowClicked` and `colClicked` by the main loop before calling this function.

You then need to complete the following in sequence:

- If the clicked tile is already revealed or flagged, do nothing and return early.
- If the clicked tile contains a mine:
  - Reveal all mines
  - Remove all flags
  - Consider the player to have lost, and return
- Call `revealTiles` (our next task) with the appropriate arguments
- Check if all tiles without mines are flagged or revealed
  - If yes, consider the player to have won and return
- Otherwise, the game should continue.

For each branch, remember to set the `curStatus` to an appropriate value before returning.

If you need some more hints, read the provided `handleRightClick` function.

4.5. Task 3: `revealTiles`

In this task, you should reveal the tiles around the input arguments recursively. The arguments `$a0` and `$a1` represent the row and the column respectively.

The algorithm is as follows:

- First, check whether the input tile is already revealed, has a flag, or is out of bounds for the board. If so, do nothing and return early.
- Reveal the input tile by setting the appropriate bit.
- If the input tile has no neighbouring mines, recursively call `revealTiles` with all 8 neighbour tiles.

A sample C++ implementation is provided in Appendix D as an example.

4.6. `handleRightClick`

This final function handles the behaviour of a right click on a given tile.

It accomplishes the following:

- If the tile is already revealed, do nothing and returns.
- Toggles the flag at the given tile (if there is already a flag, remove it. Otherwise, add a flag).
- If the user has successfully added a flag on every mine and there are no extra flags, consider the user as won.
- Otherwise, the game should continue.

As before, the function will update `curStatus` to an appropriate value before returning.

4.7. General Tips

- Remember to save the `$s` registers and the stack pointer before calling a function.
- Clever use of bit masks and bit shifting can greatly simplify your code.

5. Submission and Grading

You only need to submit `skeleton.s` on Canvas. Please write down your name, student ID and email address on the top of the skeleton code.

The deadline is 14 May 2026 (Thursday), 2359 HKT.

The deadline is a hard deadline. If multiple versions of the code are uploaded, we will by default grade your latest submission unless otherwise requested.

6. Frequently Asked Questions

Q: I am having platform issues!

A: Please first try to re-download the JRE with the instructions from Appendix A. If they still cannot be resolved, please first use Piazza to reach out so more students can benefit from your question. If your problem cannot be resolved on Piazza, feel free to email the IA-in-charge at yhccolin@ust.hk to book a meeting.

Q: Why does MARS sometimes hang / freeze if I close the game window?

A: This is due to a synchronization issue between the game window and the MIPS thread. While this should be impossible, do raise a bug report (with how to reproduce the freeze) so we can reproduce and fix this issue.

Q: Some of my inputs are ignored as if I never made them!

A: This is due to either your MIPS code being too slow, or you making inputs too quickly. Currently, the GUI discards all mouse clicks which your MIPS code cannot handle in time. If you are having this issue, check whether your code is running excessive loops.

Q: The window is not responsive to my inputs!

A: It is likely that you have an infinite loop somewhere. Use breakpoints judiciously to figure out where this infinite loop is. You might need to re-launch MARS if the entire application becomes unresponsive, though this is unlikely.

Q: My MARS is too small!

A: This is usually because of having a high-resolution monitor. You can work around this by launching the JAR with the Java argument `-Dsun.java2d.uiScale=2`, which scales everything up.

Q: I have more questions!

A: Please post them to the course Piazza if the question is not directly related to your own code. Otherwise, you can email the IA-in-charge at yhccolin@ust.hk.

7. Appendix A: Java 17 Installation

This project requires Java 17 or later. We recommend you to install the Adoptium JDK (found here) or the Amazon Correto JDK if you don't have a preference.

You only need to install the JRE; the JDK is not needed for our project.

Some platform specific notes are found below:

7.1. Windows

If you prefer an installer, you should download the `.msi` version of the JRE.

7.2. MacOS

Pay attention to what version of the JDK you are downloading; if you are using Apple Silicon (M1 or later), you should download the ARM / aarch64 versions of the JRE. Otherwise, you should download the x64 versions.

7.3. Linux

If you are using Linux, you should know what you are doing already. Please use your distribution's package manager and / or related tools to manage Java installations. Since there are so many variations, no official support will be provided. Feel free to reach out if you run into issues (the IA uses Linux as well), but help will be provided on a best-effort basis.

8. Appendix B: Extended Syscall Table

Note that default MARS syscalls are not included in this table. The extended syscalls do not use `$a1`.

Syscall	Argument (`$a0`)	Return Values		Usage
		`$v0`	`$v1`	
200 (Create Game)	-	-	-	Creates the game window.
201 (Render)	The base address of the `tiles` array.	-	-	Updates the game window to reflect the current state.
202 (Await Input)	-	The type of the input; 0 for window closing, 1 for mouse click.	-	Waits for input from the user. Inputs include mouse clicks and window closing.
203 (Get Click Type)	-	The type of the click; 0 for left clicks, 1 for right clicks.	-	Gets the mouse click type of the input.
204 (Get Click Position)	-	The clicked row.	The clicked column.	Gets the position of the clicked tile.
205 (Reset)	The current state of the game; 0 for lost, 1 for won.	Whether the user wishes to continue playing; 0 for yes, 1 for no.	-	Asks if the user wishes to continue, then resets the board if needed.
206 (Cleanup)	-	-	-	Closes any remaining game windows on exit.

9. Appendix C: Mine Locations for Some Common Seeds

Here are some mine locations for a few seeds to help with debugging.

Do note that using the "Reset" functionality will cause these to be incorrect. Please reassemble and re-run the game each time if you want to use these locations for debugging.

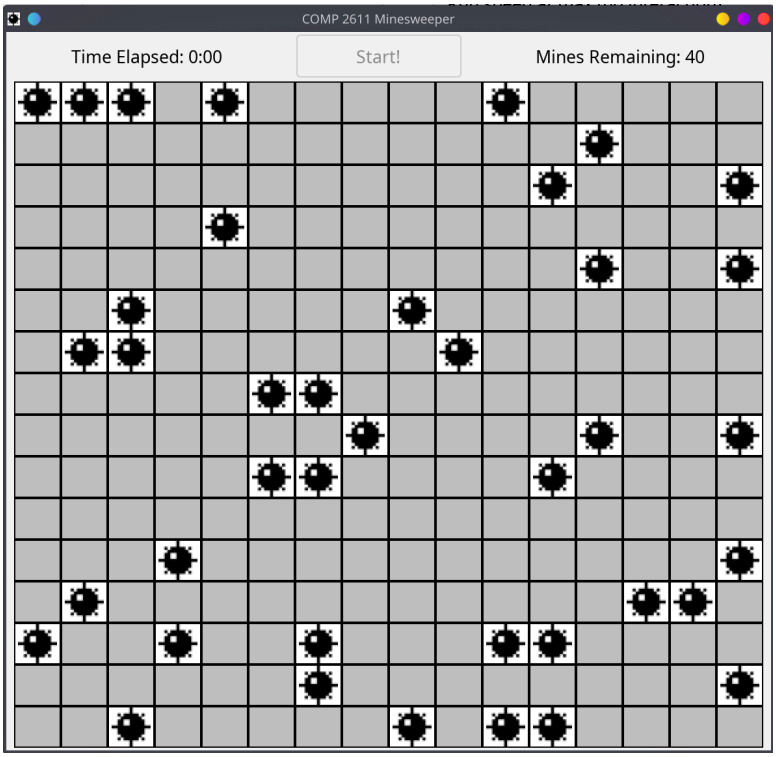

Figure 8: Mines for Seed 2611

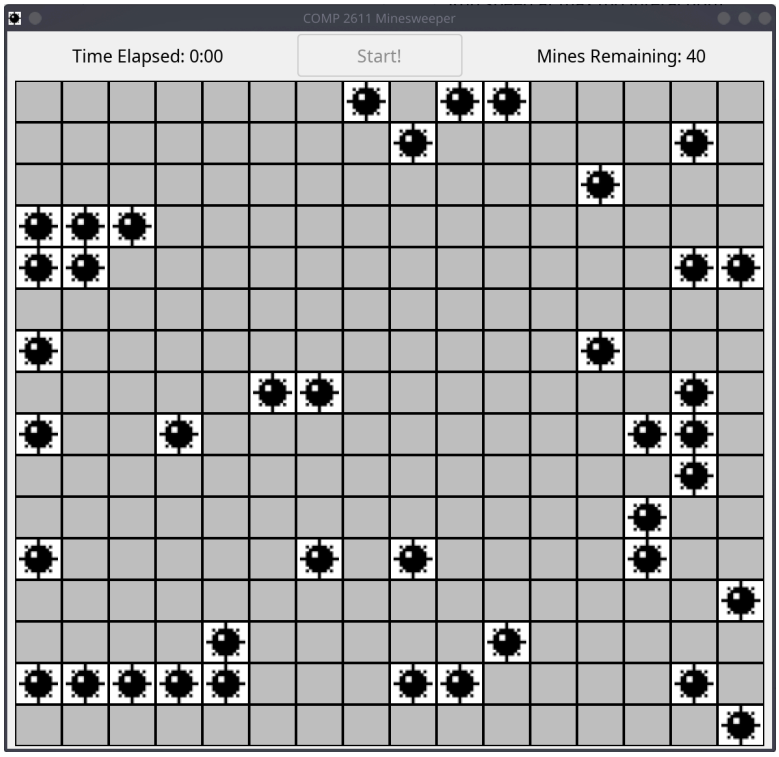

Figure 9: Mines for Seed 2012

10. Appendix D: C++ Implementation of `revealTiles`

```cpp
struct Tile {
  int row;
  int col;
  int neighbouringMines;
  bool isRevealed;
  bool hasMines;
  bool hasFlag;
};

Tile tiles[16][16];

void revealTiles(int row, int col) {
  if (row < 0 || row >= 16 || col < 0 || col >= 16) {
    return;
  }
  Tile curTile = tiles[row][col];
  if (curTile.isRevealed || curTile.hasFlag) {
    return;
  }
  curTile.isRevealed = true;
  if (curTile.neighbouringMines == 0) {
    for (int i = -1; i < 2; i++) {
      for (int j = -1; j < 2; j++) {
        revealTiles(row + i, col + j);
      }
    }
  }
}
```